

PLAN DE PRUEBAS MAESTRO (IEEE 829 / ISO 29119)

PROYECTO: AFIP — Aplicación de Finanzas Personales

VERSIÓN: 1.0

FECHA: [Fecha Actual]

AUTOR: [Líder de QA]

1. IDENTIFICACIÓN DEL PLAN DE PRUEBAS

- Identificador:** AFIP-TPM-001
- Proyecto:** AFIP — Aplicación de Finanzas Personales
- Versión del Software:** 1.0.0
- Estándar:** IEEE 829 / ISO 29119

2. INTRODUCCIÓN Y ALCANCE

2.1 Propósito

Este documento establece el plan maestro de pruebas para el sistema AFIP, definiendo la estrategia, alcance, recursos, cronograma y criterios de aceptación para validar la calidad del software.

2.2 Alcance Funcional (In-Scope)

- API Backend (Node.js + Express): Todos los endpoints REST (45 endpoints)
- Frontend Web (React): Funcionalidad, navegación, formularios
- Frontend Móvil (Flutter): Funcionalidad, navegación, formularios
- Base de Datos (MySQL): Integridad referencial, constraints
- Seguridad: Autenticación JWT, autorización por usuario
- Integridad transaccional: Operaciones que afectan múltiples tablas

2.3 Fuera de Alcance (Out-of-Scope)

- Pruebas de carga y estrés extensivas
- Pruebas de penetración profesional
- Pruebas de accesibilidad (WCAG)
- Pruebas en todos los navegadores y dispositivos posibles

3. ELEMENTOS A PROBAR

3.1 Módulos del Sistema

#	Módulo	Componente	Críticidad	Endpoints
1	Autenticación	API	Crítica	2 (register, login)
2	Usuarios	API	Crítica	5 (CRUD + initCategories)
3	Cuentas	API + Web + Móvil	Alta	5 (CRUD)
4	Categorías	API + Web + Móvil	Alta	5 (CRUD)
5	Gastos	API + Web + Móvil	Crítica	5 (CRUD transaccional)
6	Ingresos	API + Web + Móvil	Crítica	5 (CRUD transaccional + asignaciones)

7	Presupuestos	API + Web + Móvil	Alta	5 (CRUD)
8	Metas	API + Web + Móvil	Alta	5 (CRUD)
9	Deudas	API + Web + Móvil	Alta	5 (CRUD)
10	Pagos de Deuda	API + Web + Móvil	Media	5 (CRUD)
11	Middleware Auth	API	Crítica	Transversal
12	Middleware Validación	API	Alta	Transversal

4. ESTRATEGIA DE PRUEBAS

4.1 Niveles de Prueba

Nivel	Descripción	Herramientas	Responsable
Unitarias	Probar funciones/métodos individuales (controllers, middlewares)	Jest + mocks	Desarrolladores
Integración	Probar interacción entre componentes (API + BD)	Jest + Supertest	Desarrolladores
Sistema	Probar el sistema completo end-to-end	Postman / Manual	QA
Aceptación	Validar requisitos funcionales con stakeholders	Manual	QA + Product Owner

4.2 Tipos de Prueba

Tipo	Alcance	Prioridad
Funcionales	Verificar que cada endpoint cumple su especificación	Alta
Seguridad	Verificar autenticación, autorización y protección de datos	Crítica
Integridad de datos	Verificar transacciones, cascadas, constraints	Crítica
Validación	Verificar que datos inválidos son rechazados	Alta
Regresión	Verificar que cambios no rompen funcionalidad existente	Alta
Usabilidad	Verificar que la interfaz es intuitiva	Media
Compatibilidad	Verificar en diferentes navegadores/dispositivos	Media

4.3 Técnicas de Prueba

Técnica	Aplicación
Caja negra	Probar endpoints sin conocer implementación interna
Caja blanca	Probar lógica interna de controllers y middlewares
Partición de equivalencia	Agrupar datos de entrada en clases válidas e inválidas
Valores límite	Probar con valores mínimos, máximos y frontera
Casos de uso	Simular flujos completos de usuario

5. CRITERIOS DE ACEPTACIÓN

5.1 Criterios de Entrada (para iniciar pruebas)

- ☐ Código compilable sin errores
- ☐ API arranca y conecta a BD de pruebas
- ☐ Todos los endpoints accesibles
- ☐ Datos de prueba disponibles

5.2 Criterios de Salida (para aprobar las pruebas)

- ☐ 100% de casos de prueba críticos ejecutados
- ☐ 0 defectos críticos abiertos
- ☐ 0 defectos altos abiertos
- ☐ $\geq 95\%$ de casos de prueba pasaron
- ☐ Cobertura de pruebas unitarias $\geq 80\%$
- ☐ Todas las pruebas de seguridad pasaron

5.3 Criterios de Suspensión

- Más del 30% de los casos de prueba fallan
- Defecto que bloquea la ejecución de más pruebas
- Entorno de pruebas inestable

6. ENTORNO DE PRUEBAS

6.1 Configuración

Componente	Configuración de Pruebas
API	Node.js con BD de pruebas separada
Base de datos	MySQL node_project_api_test (datos de prueba)
Frontend Web	Servidor Vite dev apuntando a API de pruebas
App Móvil	Emulador/dispositivo apuntando a API de pruebas
JWT_SECRET	Clave de pruebas (diferente a producción)

6.2 Datos de Prueba

Entidad	Datos Necesarios
Usuarios	2-3 usuarios de prueba con diferentes datos
Cuentas	2-3 cuentas por usuario (diferentes tipos, saldos)
Categorías	Predefinidas (15) + 2-3 personalizadas
Gastos	5-10 gastos con diferentes categorías y cuentas
Ingresos	3-5 ingresos con y sin asignaciones
Presupuestos	2-3 presupuestos con gastos asociados
Metas	2-3 metas con diferentes niveles de progreso
Deudas	2-3 deudas (pendiente y pagada)

7. CLASIFICACIÓN DE DEFECTOS

Severidad	Descripción	Ejemplo	Tiempo de Resolución
Crítico	Sistema inoperante, pérdida de datos	Transacción no hace rollback, datos de otro usuario visibles	Inmediato (< 4h)
Alto	Funcionalidad principal no funciona	No se puede crear un gasto, login falla	< 24h
Medio	Funcionalidad secundaria afectada	Formato incorrecto de fecha, cálculo de porcentaje erróneo	< 72h
Bajo	Problema cosmético o menor	Mensaje de error en inglés en vez de español	Sprint siguiente

8. ROLES Y RESPONSABILIDADES

Rol	Responsabilidades
Líder de QA	Planificación, seguimiento, reportes de pruebas
Tester	Diseño y ejecución de casos de prueba
Desarrollador Backend	Tests unitarios del API, corrección de defectos
Desarrollador Frontend	Tests de componentes, corrección de defectos UI
Desarrollador Móvil	Tests de widgets Flutter, corrección de defectos

9. CRONOGRAMA DE PRUEBAS

Fase	Actividad	Duración	Semana
1	Preparar entorno y datos de prueba	2 días	S1
2	Pruebas unitarias del API (Jest)	3 días	S1
3	Pruebas de integración API + BD	2 días	S2
4	Pruebas funcionales Web	3 días	S2
5	Pruebas funcionales Móvil	3 días	S3
6	Pruebas de seguridad	2 días	S3
7	Pruebas de regresión	1 día	S4
8	Corrección de defectos	3 días	S4
9	Pruebas de aceptación	2 días	S4
10	Reporte final	1 día	S4

10. RIESGOS DE LAS PRUEBAS

Riesgo	Mitigación
Entorno de pruebas no disponible	Preparar scripts de setup automatizado
Datos de prueba inconsistentes	Usar fixtures/seeds con datos conocidos

Cobertura insuficiente	Definir casos de prueba antes de codificar
Falta de tiempo	Priorizar pruebas de módulos críticos

11. ENTREGABLES

Entregable	Descripción
Plan de pruebas maestro	Este documento
Plan de pruebas detallado	Casos de prueba por módulo
Código de tests unitarios	Suite de Jest con todos los tests
Reporte de ejecución	Resultados de pruebas con métricas
Lista de defectos	Defectos encontrados con severidad y estado